STUDENT NAME :

- NUSRIN FATHIMA A
- DIVYA SRI S
- DEVASRI K
- JAYAPRABA U
- SWETHA V

REGISTER NUMBER :

- 23126ER018
- 23126ER004
- 23126ER005
- 23126ER006
- 23126ER016

INSTITUTION: SRI ADI CHUNHANAGIRI WOMEN'S COLLEGE

DEPARTMENT: B.Sc., COMPUTER SCIENCE

DATE OF SUBMISSION: 08/03/2026

GITHUB REPOSITORY LINK: https://github.com/nusrin-fathima123/NM_SQL-GENERATOR/blob/main/sql_generator%20(1).py

SQL Query Generator using Natural Language Processing

Project Description

The **SQL Query Generator** is an intelligent system that converts **natural language requests into SQL queries** automatically. Many users find SQL difficult to write manually. This project solves that problem by allowing users to type their request in **plain English**, and the system generates the corresponding SQL query.

The system uses the **IBM Granite 3.3 2B Instruct model** through the Hugging Face Transformers library to understand the meaning of the user's input. Based on the detected keywords, the system identifies the correct database table and generates a valid SQL query.

The application includes a **security validation system** that prevents SQL injection attacks by blocking dangerous SQL keywords and suspicious patterns. It also supports **PDF report generation**, allowing users to export the generated SQL queries for documentation or sharing purposes.

A **Gradio-based chatbot interface** provides an interactive environment where users can enter requests and receive generated SQL queries instantly.

This project demonstrates how **Artificial Intelligence and Natural Language Processing can simplify database querying** for users who do not have technical knowledge of SQL.

Project Objectives

The main objectives of this project are:

• To convert natural language queries into SQL statements.
• To simplify database querying for non-technical users.
• To prevent SQL injection attacks through security validation.
• To provide an interactive chatbot interface for query generation.
• To allow exporting generated SQL queries as PDF reports.

Abstract

The **SQL Query Generator** is a system designed to convert natural language queries into SQL statements automatically. Many users find it difficult to write SQL queries manually, especially without technical knowledge of databases. This project simplifies the process by allowing users to enter requests in plain English, and the system generates the corresponding SQL query.

The system uses the **IBM Granite 3.3 2B Instruct model** with Python and the Transformers library to understand user input. It detects relevant tables and conditions from the request and generates an appropriate SQL query. A **security validation module** is included to prevent SQL injection by blocking dangerous keywords and suspicious patterns.

The application provides an interactive **Gradio-based chat interface** where users can enter their requests and receive generated SQL queries instantly. It also includes a **PDF export feature** that allows users to save generated queries as reports.

This project demonstrates how **Artificial Intelligence and Natural Language Processing can make database querying easier and more accessible for users without SQL expertise.**

Introduction

The **SQL Query Generator using Natural Language** is a system designed to convert user queries written in simple English into SQL queries. Many users find it difficult to write SQL commands because it requires knowledge of database syntax and structure. This system helps overcome that problem by allowing users to enter their requests in natural language.

The system analyzes the user's input, identifies keywords related to database tables and conditions, and then generates the corresponding SQL query automatically. It also includes features such as input validation, query history storage, and PDF export of generated queries.

This project improves accessibility to databases by making query generation easier, faster, and more user-friendly, especially for beginners who are not familiar with SQL syntax.

Acknowledgement

I would like to express my sincere gratitude to my project guide and faculty members for their valuable guidance, support, and encouragement throughout the development of this project. Their suggestions and feedback helped me improve the quality of this work.

I would also like to thank my institution for providing the necessary resources and environment to complete this project successfully. Finally, I extend my heartfelt thanks to my friends and family for their constant support and motivation during the completion of this project.

Technologies Used

Programming Language

Python 3.x

Libraries

Transformers – for loading the IBM Granite language model
Torch – for running the AI model
Gradio – for creating the chat interface
ReportLab – for generating PDF reports
Regex – for text pattern matching

Platform

Google Colab / Python Environment

System Architecture

The system consists of several main components:

1. **User Interface**

2. **Natural Language Processing Engine**

3. **Security Validation System**

4. **SQL Query Generator**

5. **Export System**

Workflow:

User Input → Security Validation → Table Detection → Condition Extraction → SQL Generation → Display Result → Export Option

Database Schema

The system works with four tables.

Users Table

Column	Data Type
id	INTEGER
name	TEXT
email	TEXT
signup_date	DATE
age	INTEGER
country	TEXT
status	TEXT

The **Users table** stores information about all registered users in the system. It includes details such as user ID, name, email address, signup date, age, country, and account status. This table helps the system manage and retrieve user-related data when generating SQL queries based on the user's natural language request.

Orders Table

Column	Data Type
id	INTEGER
user_id	INTEGER
product_name	TEXT
amount	REAL
order_date	DATE
status	TEXT

Products Table

Column	Data Type
id	INTEGER
name	TEXT
price	REAL
category	TEXT
stock	INTEGER

Transactions Table

Column	Data Type
id	INTEGER
user_id	INTEGER
amount	REAL
date	DATE
type	TEXT

Data Flow Diagram (DFD)

A **Data Flow Diagram (DFD)** shows how data moves through the system and how it is processed. In the SQL Query Generator system, the user provides a natural language request, which is processed by different modules to generate a SQL query.

DFD Explanation

1. **User Input**
   The user enters a request in natural language through the chat interface.

2. **Input Validation Module**
   The system checks the input for dangerous SQL keywords or injection patterns to ensure security.

3. **Natural Language Processing Module**
   The system analyzes the input and detects keywords related to database tables and conditions.

4. **SQL Query Generator**
   Based on the detected table and conditions, the system generates the appropriate SQL query.

5. **Output Display**
   The generated SQL query is displayed to the user through the Gradio chat interface.

6. **Export Module**
   The generated query can be saved as a PDF report for documentation or sharing

7.
```
User
  |
  v
User Interface (Gradio Chat)
  |
  v
Input Validation
  |
  v
Natural Language Processing
  |
  v
SQL Query Generator
  |
  v
Generated SQL Query
  |
  v
PDF Export / Output Display
```

System Modules

1. Model Initialization

The IBM Granite model is loaded using the Hugging Face Transformers library.

The tokenizer converts user input into tokens and the model generates a response based on the prompt.

This allows the system to understand natural language requests.

2. Security Validation Module

To prevent malicious queries, the system checks user input using the **is_safe() function**.

The system blocks dangerous SQL keywords such as:

DROP
DELETE
ALTER
TRUNCATE
INSERT
UPDATE

It also detects injection patterns like:

;

/* */
xp_
sp_

If any unsafe pattern is detected, the system blocks the request and displays an error message.

3. Table Detection Module

The **get_table_name() function** identifies which table should be used for the query.

Example keyword mapping:

User related words → users table
Order related words → orders table
Product related words → products table
Transaction related words → transactions table

This helps the system select the correct database table automatically.

4. Condition Extraction Module

The **extract_conditions() function** analyzes the user's request and extracts filtering conditions.

Examples:

greater than 500 → amount > 500

last week → date >= last 7 days

last month → signup_date >= last 1 month

These conditions are added to the SQL WHERE clause.

5. SQL Query Generation Module

The **generate_sql() function** builds the final SQL query.

Steps performed:

1. Validate input security
2. Detect table name
3. Extract conditions
4. Construct SQL query
5. Add LIMIT clause

Example Output:

```
SELECT * FROM users
WHERE DATE(signup_date) >= DATE('now','-1 month')
LIMIT 10;
```

6. Chat Interface Module

The system uses **Gradio Blocks interface** to create an interactive chatbot.

Features include:

• Chat-style interface
• Text input box
• Automatic SQL generation
• Conversation history

Users can type their request and instantly receive the generated SQL query.

7. PDF Export Module

The **ReportLab library** is used to generate a professional PDF report.

The exported PDF contains:

• User queries
• Generated SQL statements
• Timestamp
• Report title

This allows users to store or share the generated queries.

System Design

1. User Interface (Gradio)
   Provides a chat-based interface where users can enter natural language queries.

2. Input Validation Module
   Checks the user input for unsafe SQL keywords or injection patterns to ensure system security.

3. Natural Language Processing Module
   Analyzes the user request and identifies the relevant table and conditions using keyword detection.

4. SQL Query Generator
   Constructs the SQL query based on detected tables and conditions.

5. Query History Storage
   Stores the generated queries for future reference and exporting.

6. PDF Export Module
   Generates a PDF report containing the user queries and generated SQL statements.

```
User
  |
  v
Gradio User Interface
  |
  v
Input Validation
  |
  v
Natural Language Processing
  |
  v
SQL Query Generator
  |
  v
Query History
  |
  v
PDF Export / Output Display
```

Project Workflow

Step 1: Install Dependencies

Required libraries are installed using pip.

pip install gradio transformers torch reportlab requests

Step 2: Load AI Model

The IBM Granite model is loaded from Hugging Face.

Step 3: Define Database Schema

Tables and column names are defined using Python dictionaries.

Step 4: Implement Security System

Dangerous keywords and injection patterns are blocked.

Step 5: Develop SQL Generation Engine

Functions are created for:

• Table detection
• Condition extraction
• SQL query generation

Step 6: Create Gradio Interface

A chatbot interface is developed for user interaction.

Step 7: Implement Export Feature

Generated queries are saved as PDF reports.

Sample Test Cases

Test Case 1

Input:

Show users who signed up last month

Output:

```
SELECT * FROM users
WHERE DATE(signup_date) >= DATE('now','-1 month')
LIMIT 10
```

Test Case 2

Input:

Find transactions greater than 500 from last week

Output:

```
SELECT * FROM transactions
WHERE amount > 500 AND DATE(date) >= DATE('now','-7 days')
LIMIT 10
```

Test Case 3

Input:

Show product details

Output:

```
SELECT * FROM products
LIMIT 10
```

Block Diagram

The **Block Diagram** shows the main components of the SQL Query Generator system and how data flows between them. It illustrates the process from user input to the generation of an SQL query.

Main Blocks

1. **User Input** – The user enters a query in natural language.

2. **User Interface (Gradio)** – Provides the interface to submit the request.

3. **Input Validation** – Checks the input for unsafe SQL keywords or errors.

4. **Natural Language Processing** – Identifies keywords, tables, and conditions from the input.

5. **SQL Query Generator** – Converts the processed text into an SQL query.

6. **Output Display / Query History** – Shows the generated SQL query and stores it for reference.

```
User
  |
  v
User Interface (Gradio)
  |
  v
Input Validation
  |
  v
Natural Language Processing
  |
  v
SQL Query Generator
  |
  v
Output Display / Query History
```

Advantages of the System

- Easy to use interface
- Reduces the need to learn SQL syntax
- Saves time in writing queries
- Improves accessibility to databases
- Stores query history for reference

Limitations

- Works with limited predefined tables
- May not understand very complex queries
- Requires proper keyword matching

Future Enhancements

Future improvements can include:

• Support for JOIN queries
• Support for GROUP BY and aggregation
• Connection to real databases (MySQL, PostgreSQL)
• More advanced natural language understanding
• Web application deployment

Conclusion

The SQL Query Generator project successfully demonstrates how **Natural Language Processing and Artificial Intelligence can simplify database querying**. By allowing users to interact with the system using plain English, the project eliminates the need for advanced SQL knowledge.

The integration of the IBM Granite model, security validation mechanisms, and a user-friendly Gradio interface makes the system both powerful and easy to use. The addition of PDF export functionality further enhances the usability of the system for reporting and documentation purposes.

This project highlights the potential of AI-driven tools in improving productivity and accessibility in database management systems.